

# UNIVERSIDAD CONTINENTAL

## FACULTAD DE INGENIERIA

ESCUELA ACADEMICO PROFESIONAL DE INGENIERIA DE SISTEMAS E INFORMATICA

# PLAN DE PRUEBAS DEL SISTEMA WEB DE VALORIZACION DE OBRA

*Incluye implementacion de pruebas de carga con k6 y analisis de calidad con SonarQube*

| Dato                       | Descripcion                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Proyecto                   | Sistema web con enfoque BPM y apoyo de inteligencia artificial para la elaboracion del informe mensual de valorizacion de obra. |
| Repositorio base           | valorizacion-obra-web-main.zip                                                                                                  |
| Herramientas implementadas | k6 para carga/rendimiento y SonarQube/SonarCloud para calidad de codigo.                                                        |
| Version del plan           | v2.0 - Actualizado despues de implementar k6 y SonarQube en el proyecto.                                                        |
| Fecha de elaboracion       | Junio de 2026                                                                                                                   |

Autores del informe base: Rodriguez Rios Nathalie Tatiana, Eufrazio Abal Leonardo Paolo, Gamarra Huaranga Grace Malu.

Asesor: Dr. Maglioni Arana Caparachin.

## Control de cambios

| Version | Cambio realizado                                                      | Evidencia                            |
|---------|-----------------------------------------------------------------------|--------------------------------------|
| 1.0     | Plan inicial de pruebas del sistema de valorizacion de obra.          | Documento previo del plan.           |
| 2.0     | Implementacion de k6, SonarQube, workflows CI y scripts de preflight. | Repositorio modificado y nuevo plan. |

## Resumen de implementacion tecnica realizada

Se agregaron archivos de configuracion, escenarios de carga, pipelines de CI y pruebas de cumplimiento para que el repositorio tenga una implementacion verificable de k6 y SonarQube. La implementacion se diseno para no usar datos productivos y para ejecutarse contra preview local o ambiente QA.

| Archivo implementado/modificado            | Funcion dentro del plan de pruebas                                                  |
|--------------------------------------------|-------------------------------------------------------------------------------------|
| sonar-project.properties                   | Configura SonarQube/SonarCloud, fuentes, pruebas, exclusiones, LCOV y Quality Gate. |
| .github/workflows/quality-sonarqube.yml    | Ejecuta dependencias, preflight, cobertura, build y analisis SonarQube en CI.       |
| .github/workflows/k6-load-tests.yml        | Permite ejecutar smoke, carga, estres, pico, resistencia y lectura Supabase en CI.  |
| tests/k6/shared/config.js                  | Centraliza URL, umbrales, escenarios y tiempos de espera.                           |
| tests/k6/shared/helpers.js                 | Define navegacion SPA, probes Supabase y validaciones no destructivas.              |
| tests/k6/scenarios/smoke.js                | Smoke test: 1 usuario virtual durante 1 minuto.                                     |
| tests/k6/scenarios/load.js                 | Carga esperada: incremento progresivo hasta 25 usuarios virtuales.                  |
| tests/k6/scenarios/stress.js               | Estres: incremento controlado hasta 75 usuarios virtuales.                          |
| tests/k6/scenarios/spike.js                | Pico: subida rapida hasta 60 usuarios virtuales.                                    |
| tests/k6/scenarios/soak.js                 | Resistencia: 10 usuarios virtuales durante 30 minutos.                              |
| tests/k6/scenarios/supabase-read.js        | Probes de lectura no destructiva para Supabase QA.                                  |
| scripts/run-k6.sh                          | Ejecutor local de k6, con fallback a Docker.                                        |
| scripts/sonar-scan.sh                      | Ejecutor local de SonarScanner, con fallback a Docker.                              |
| scripts/verify-k6-sonar.mjs                | Preflight para comprobar que k6 y SonarQube sigan configurados.                     |
| src/lib/quality-tooling/compliance.test.ts | Prueba automatizada que valida archivos y scripts de calidad.                       |
| vitest.config.ts                           | Configura cobertura V8 y reportes LCOV para SonarQube.                              |
| package.json                               | Agrega scripts npm de cobertura, k6, SonarQube y calidad.                           |
| package-lock.json                          | Actualizado para sincronizar dependencias y cobertura Vitest.                       |
| docs/IMPLEMENTACION-K6-SONARQUBE.md        | Documenta variables, comandos, umbrales y criterios.                                |
| docs/PLAN-PRUEBAS.md                       | Actualiza la matriz de pruebas interna del repositorio.                             |

Verificacion local realizada: se ejecuto el preflight de configuracion ``npm run quality:preflight``, que valida la existencia de k6, SonarQube, scripts npm, workflows y umbrales. Las pruebas k6 reales requieren el binario k6 o Docker; quedaron listas para ejecutarse en CI o en una maquina con k6 instalado.

# 1. Introduccion

## 1.1. Proposito del documento

El proposito de este documento es definir el plan de pruebas del sistema web de valorizacion de obra, incorporando la implementacion tecnica realizada para pruebas de carga con k6 y analisis de calidad con SonarQube. El plan permite organizar pruebas funcionales, no funcionales, de seguridad, rendimiento, calidad de codigo, trazabilidad documental e inteligencia artificial responsable.

## 1.2. Alcance

El plan cubre el flujo principal del sistema: autenticacion, gestion de proyectos, carga de linea base, presupuesto, partidas, metrados, evidencias, fiscalizacion del supervisor, valorizacion, reajustes, amortizaciones, generacion documental asistida por Claude, exportacion Word/PDF, firma interna y auditoria. Tambien cubre la automatizacion de pruebas mediante Vitest, Playwright, k6, SonarQube y GitHub Actions.

## 1.3. Definiciones, acronimos y referencias

| Termino      | Definicion                                                                                         |
|--------------|----------------------------------------------------------------------------------------------------|
| BPM          | Gestion de procesos de negocio usada para modelar el flujo AS-IS y TO-BE de valorizacion.          |
| k6           | Herramienta de pruebas de carga y rendimiento basada en scripts JavaScript.                        |
| SonarQube    | Plataforma de analisis estatico para calidad, deuda tecnica, bugs, vulnerabilidades y duplicacion. |
| LCOV         | Formato de reporte de cobertura consumido por SonarQube para proyectos JavaScript/TypeScript.      |
| Quality Gate | Conjunto de reglas de calidad que debe aprobar el codigo antes de despliegue.                      |
| p95 / p99    | Percentiles de tiempo de respuesta usados para evaluar rendimiento.                                |
| VU           | Usuario virtual simulado por k6.                                                                   |
| QA           | Ambiente de pruebas con datos ficticios y roles preconfigurados.                                   |

# 2. Objetivos del Plan de Pruebas

## 2.1. Objetivos generales

- Validar que el sistema soporte el proceso completo de elaboracion del informe mensual de valorizacion de obra.
- Comprobar la exactitud de calculos de metrados, valorizacion, reajustes, amortizaciones y saldos.
- Asegurar que los roles Residente, Supervisor, Entidad, Representante Legal y Administrador tengan permisos correctos.
- Medir el rendimiento del sistema bajo carga esperada mediante k6.
- Controlar la mantenibilidad, seguridad, duplicacion y deuda tecnica mediante SonarQube.
- Garantizar que la IA genere borradores editables sin reemplazar la revision humana.

## 2.2. Resultados esperados

| Resultado esperado                 | Indicador verificable                          | Herramienta             |
|------------------------------------|------------------------------------------------|-------------------------|
| Flujo funcional completo aprobado  | Casos P0/P1 sin defectos criticos abiertos     | Vitest, Playwright, UAT |
| Calculos economicos confiables     | Diferencia cero frente a golden data definido  | Vitest                  |
| Rendimiento aceptable              | p95 <= 3 s, p99 <= 5 s, errores < 1%           | k6                      |
| Calidad de codigo controlada       | Quality Gate aprobado                          | SonarQube/SonarCloud    |
| Cobertura disponible para analisis | Archivo coverage/lcov.info generado            | Vitest/V8               |
| Trazabilidad documental            | Version, hash, usuario y fecha por exportacion | Sistema + auditoria     |

## 3. Alcance de las Pruebas

### 3.1. Funcionalidades a probar

| Modulo                 | Funcionalidades incluidas                                                                |
|------------------------|------------------------------------------------------------------------------------------|
| Autenticacion y roles  | Login, rutas protegidas, RBAC, restriccion de alta publica y perfiles de usuario.        |
| Proyectos y linea base | Ficha tecnica, contrato, presupuesto base, partidas, cronograma, indices y adelantos.    |
| Metrados y evidencias  | Registro por partida, formulas/dimensiones, validacion contra saldo, fotos y documentos. |
| Fiscalizacion          | Envio a revision, observacion, recorte, levantamiento, aprobacion y cierre.              |
| Calculo economico      | Valorizacion anterior, actual, acumulada, saldo, factor K, reajustes y amortizaciones.   |
| Generacion documental  | Borradores con Claude, revision humana, exportacion Word/PDF, hash y version.            |
| Calidad de codigo      | Analisis estatico, cobertura, duplicacion, deuda tecnica y Quality Gate.                 |
| Rendimiento            | Navegacion SPA, rutas principales, probes Supabase QA y respuesta bajo concurrencia.     |

### 3.2. Funcionalidades fuera de alcance

- Integracion oficial con SEACE, ReFirma, S10 o ERP externo.
- Firma digital certificada oficial; se valida solo firma electronica interna del MVP.
- Pruebas destructivas o de escritura masiva contra produccion.
- Reconocimiento automatico de imagenes o validacion visual de fotografias mediante IA.
- Liquidacion final con validez legal definitiva fuera del expediente mensual.

## 4. Estrategia de Pruebas

### 4.1. Enfoque general

La estrategia es progresiva, basada en riesgos y automatizable. Primero se validan funciones puras y reglas de negocio; luego componentes, integraciones, flujos E2E, seguridad, rendimiento y aceptacion de usuario. Las pruebas de carga y calidad de codigo se integran al pipeline para evitar que un cambio llegue a release sin evidencia.

### 4.2. Niveles de prueba

| Nivel             | Objetivo                                                      | Herramienta                    |
|-------------------|---------------------------------------------------------------|--------------------------------|
| Unitarias         | Validar calculos, politicas y transformaciones de datos.      | Vitest                         |
| Componentes       | Validar formularios, tablas, mensajes y estados de UI.        | Vitest / React Testing Library |
| Integracion       | Validar relacion frontend, Supabase, storage, auditoria e IA. | Vitest, pruebas API, mocks     |
| Sistema           | Validar flujo completo por roles.                             | Playwright                     |
| Aceptacion        | Validar utilidad con usuarios representativos.                | Checklists UAT                 |
| Carga/rendimiento | Medir respuesta bajo concurrencia y picos.                    | k6                             |
| Calidad estatica  | Detectar deuda, duplicacion, vulnerabilidades y bugs.         | SonarQube                      |

### 4.3. Tipos de pruebas

| Tipo        | Criterio principal                                        | Evidencia                    |
|-------------|-----------------------------------------------------------|------------------------------|
| Funcionales | Requerimientos RF aprobados.                              | Casos, capturas, reportes.   |
| Seguridad   | Sin vulnerabilidades criticas/altas y permisos correctos. | CodeQL, Gitleaks, ZAP, RBAC. |
| Rendimiento | p95/p99 y tasa de error dentro de umbrales.               | Reportes k6.                 |
| Usabilidad  | Usuarios completan tareas criticas sin bloqueo.           | Encuesta y UAT.              |

|                   |                                                       |                            |
|-------------------|-------------------------------------------------------|----------------------------|
| Regresion         | Cambios no rompen flujos existentes.                  | Vitest, Playwright, CI.    |
| IA responsable    | Claude no inventa datos y requiere aprobacion humana. | Matriz IA, logs y prompts. |
| Calidad de codigo | Quality Gate aprobado.                                | Dashboard SonarQube.       |

## 5. Entorno de Pruebas

### 5.1. Hardware

| Entorno           | Hardware minimo sugerido                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Desarrollo local  | Equipo con 8 GB RAM, CPU 4 nucleos, SSD y Node.js LTS.                                              |
| QA                | Servidor o servicio cloud con recursos equivalentes al MVP, logs habilitados y datos ficticios.     |
| CI GitHub Actions | Runner ubuntu-latest, Node.js 22 y cache npm.                                                       |
| Carga k6          | Maquina local con k6 o runner con Docker; no ejecutar cargas altas desde equipos de baja capacidad. |

### 5.2. Software

| Elemento      | Herramienta/configuracion                          |
|---------------|----------------------------------------------------|
| Frontend      | React, TanStack Start, Vite, Tailwind y shadcn/ui. |
| Base de datos | Supabase PostgreSQL, Auth, Storage y RLS.          |
| Unitarias     | Vitest con cobertura V8.                           |
| E2E           | Playwright.                                        |
| Carga         | k6 con escenarios JS.                              |
| Calidad       | SonarQube/SonarCloud con LCOV.                     |
| CI/CD         | GitHub Actions.                                    |
| Seguridad     | CodeQL, Gitleaks, OWASP ZAP y reglas RBAC.         |

### 5.3. Datos de prueba

| Dato          | Descripcion                                                                  |
|---------------|------------------------------------------------------------------------------|
| Obras semilla | Obra por precios unitarios con adelantos y obra a suma alzada sin adelantos. |
| Presupuesto   | Partidas con unidad, metrado contractual, precio unitario y parcial.         |
| Metrados      | Casos validos, excedentes, decimales extremos y formulas invalidas.          |
| Evidencias    | Fotos y ensayos ficticios vinculados a partida/periodo.                      |
| Usuarios      | Administrador, Residente, Supervisor, Entidad y Representante Legal.         |
| Estados       | Abierto, en revision, observado, corregido, aprobado, cerrado y exportado.   |
| k6            | URL QA/preview, Supabase anon key QA opcional y token QA opcional.           |

### 5.4. Configuracion del entorno

| Archivo/variable                 | Uso                                                        |
|----------------------------------|------------------------------------------------------------|
| .env.k6.example                  | Plantilla de variables de carga sin credenciales reales.   |
| BASE_URL                         | URL de la aplicacion en local, QA o produccion controlada. |
| SUPABASE_URL / SUPABASE_ANON_KEY | Lecturas no destructivas de Supabase QA.                   |
| K6_AUTH_TOKEN                    | Token opcional de QA para rutas protegidas.                |
| SONAR_TOKEN / SONAR_HOST_URL     | Secretos de CI para SonarQube/SonarCloud.                  |

## 6. Recursos

### 6.1. Equipo de trabajo, roles y responsabilidades

| Rol           | Responsabilidades                                                                |
|---------------|----------------------------------------------------------------------------------|
| Lider tecnico | Define criterios, revisa PR, valida SonarQube y aprueba calidad tecnica.         |
| QA            | Diseña casos, ejecuta pruebas, registra defectos, consolida evidencias k6 y UAT. |
| Desarrollador | Implementa correcciones, mantiene pruebas unitarias y evita regresiones.         |

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| DevSecOps       | Configura GitHub Actions, secretos, SonarQube, k6 y escaneos de seguridad. |
| Residente       | Valida flujo de registro de metrados, evidencias y generacion documental.  |
| Supervisor      | Valida observaciones, recortes, aprobaciones y cierre de periodo.          |
| Entidad/Monitor | Valida acceso de solo lectura y descarga de expedientes aprobados.         |

## 6.2. Herramientas de testing

| Herramienta          | Uso en el proyecto                                           |
|----------------------|--------------------------------------------------------------|
| Vitest               | Unitarias, integracion y cobertura LCOV.                     |
| Playwright           | Flujos E2E de login, rutas protegidas y recorrido principal. |
| k6                   | Carga, stres, pico, resistencia y smoke de rendimiento.      |
| SonarQube/SonarCloud | Quality Gate, deuda, bugs, vulnerabilidades y duplicacion.   |
| GitHub Actions       | Ejecucion automatica de calidad, build, Sonar y k6.          |
| CodeQL/Gitleaks/ZAP  | Seguridad estatica, secretos y DAST basico.                  |

## 6.3. Infraestructura necesaria

- Repositorio GitHub con Actions habilitado.
- Secretos configurados: SONAR\_TOKEN, SONAR\_HOST\_URL, QA\_SUPABASE\_URL, QA\_SUPABASE\_ANON\_KEY y, si aplica, QA\_K6\_AUTH\_TOKEN.
- Ambiente QA con datos ficticios y sin informacion sensible real.
- Reglas de Cloudflare o infraestructura equivalente para WAF, TLS y rate limiting.

# 7. Planificacion y Cronograma

## 7.1. Fases de pruebas

| Fase           | Actividad                                                       | Salida                    |
|----------------|-----------------------------------------------------------------|---------------------------|
| F1 Preparacion | Definir matriz, ambientes, usuarios y datos semilla.            | Ambiente QA listo.        |
| F2 Unitarias   | Ejecutar logica critica, roles, seguridad y cobertura.          | Reporte Vitest/LCOV.      |
| F3 Integracion | Validar Supabase, storage, IA, exportacion y auditoria.         | Reporte integracion.      |
| F4 E2E         | Validar flujo multirol de valorizacion.                         | Reporte Playwright.       |
| F5 Sonar       | Ejecutar Quality Gate con cobertura.                            | Dashboard Sonar aprobado. |
| F6 k6          | Ejecutar smoke, carga esperada y, antes de release, stres/pico. | Reporte p95/p99/errores.  |
| F7 UAT         | Validar con Residente, Supervisor y Entidad.                    | Acta de aceptacion.       |

## 7.2. Fechas clave

| Semana   | Actividad principal                        | Hito                 |
|----------|--------------------------------------------|----------------------|
| Semana 1 | Preparacion de datos, roles y ambiente QA. | Matriz aprobada.     |
| Semana 2 | Unitarias, cobertura y SonarQube inicial.  | Primer Quality Gate. |
| Semana 3 | E2E, integracion y pruebas documentales.   | Flujo P0 aprobado.   |
| Semana 4 | k6 smoke/carga y seguridad.                | Reporte rendimiento. |
| Semana 5 | UAT, correcciones y regresion.             | Acta de aceptacion.  |

## 7.3. Hitos

- H1: preflight k6/SonarQube aprobado.
- H2: cobertura LCOV generada y consumida por SonarQube.
- H3: Quality Gate aprobado en CI.
- H4: k6 smoke aprobado en preview/QA.
- H5: k6 carga esperada aprobado antes de release.
- H6: UAT firmado por usuarios clave.

## 8. Criterios de Entrada y Salida

### 8.1. Condiciones para iniciar pruebas

- Repositorio actualizado con dependencias sincronizadas.
- Ambiente QA disponible con datos ficticios y roles configurados.
- Variables de entorno definidas para k6 y SonarQube.
- Matriz de requerimientos y casos P0/P1 aprobada.
- Sin despliegues de carga contra produccion salvo smoke controlado.

### 8.2. Condiciones para finalizar pruebas

- 100% de casos P0 aprobados.
- Sin defectos criticos o altos abiertos.
- Quality Gate aprobado o defectos Sonar justificados y corregidos antes de release.
- k6 smoke y carga esperada dentro de umbrales.
- Expediente Word/PDF exportado con version, hash y trazabilidad.
- Acta UAT o reporte final de pruebas firmado.

## 9. Casos de Prueba

### 9.1. Metodologia para disenarlos

Los casos se diseñan por riesgo y trazabilidad. Cada requerimiento critico se vincula a un caso funcional, una tecnica de prueba, un dato de prueba, un responsable, una evidencia y un criterio de aceptacion. Las pruebas automatizadas se ejecutan en CI, mientras que UAT valida utilidad operativa.

### 9.2. Trazabilidad con requisitos

| Req.          | Modulo                          | Pruebas asociadas                    | Evidencia                      |
|---------------|---------------------------------|--------------------------------------|--------------------------------|
| RF-01 a RF-03 | Usuarios, roles y proyectos     | Roles, seguridad, E2E                | Reporte RBAC, capturas y logs. |
| RF-04 a RF-08 | Presupuesto, partidas e indices | Unitarias, importacion, datos limite | Reporte Vitest y dataset.      |
| RF-09 a RF-16 | Periodos, metrados y evidencias | Funcionales, E2E, storage            | Capturas y registros BD.       |
| RF-17 a RF-21 | Fiscalizacion                   | E2E multirol, regresion              | Historial workflow.            |
| RF-22 a RF-27 | Valorizacion y reajustes        | Golden data, unitarias               | Comparacion patron.            |
| RF-28 a RF-33 | Claude y expediente             | IA responsable, documental           | Word/PDF, prompt y hash.       |
| RNF-02        | Rendimiento                     | k6 smoke, load, stress               | p95/p99, errores, checks.      |
| RNF-10        | Calidad de codigo               | SonarQube + LCOV                     | Quality Gate.                  |

### 9.3. Catalogo de casos principales

| ID        | Caso                                | Herramienta       | Resultado esperado                             |
|-----------|-------------------------------------|-------------------|------------------------------------------------|
| AUTH-001  | Acceder a login.                    | Playwright/Vitest | No existe alta publica visible.                |
| ROLE-001  | Residente crea proyecto.            | Vitest/E2E        | Operacion permitida y auditada.                |
| MET-001   | Registrar metrado valido.           | Vitest/UAT        | Parcial y acumulado correctos.                 |
| MET-002   | Intentar exceder saldo contractual. | Vitest/E2E        | Sistema bloquea o exige modificacion aprobada. |
| VAL-001   | Calcular valorizacion mensual.      | Vitest            | Montos coinciden con golden data.              |
| REA-001   | Calcular factor K y reajuste.       | Vitest            | Resultado coincide con patron.                 |
| DOC-001   | Exportar expediente.                | UAT/regresion     | Word/PDF con secciones, version y hash.        |
| IA-001    | Generar borrador Claude.            | Matriz IA         | Texto no inventa datos y queda editable.       |
| SONAR-001 | Ejecutar Quality Gate.              | SonarQube         | Analisis aprobado.                             |
| K6-001    | Ejecutar smoke.                     | k6                | Sin 5xx, checks > 99%.                         |
| K6-002    | Ejecutar carga esperada.            | k6                | p95 <= 3 s, p99 <= 5 s, errores < 1%.          |

### 9.4. Casos k6 implementados

| Escenario      | Usuarios virtuales | Duracion/etapas                  | Criterio                            |
|----------------|--------------------|----------------------------------|-------------------------------------|
| Smoke          | 1 VU               | 1 minuto                         | Disponibilidad y respuesta sin 5xx. |
| Carga esperada | Hasta 25 VU        | 2m -> 5, 4m -> 25, 4m sostenido, | p95 <= 3s, p99 <= 5s, errores <     |

|               |              |                          |                                               |
|---------------|--------------|--------------------------|-----------------------------------------------|
|               |              | 2m bajada                | 1%.                                           |
| Estres        | Hasta 75 VU  | 25 -> 50 -> 75 VU        | Identificar limite sin caída total.           |
| Pico          | Hasta 60 VU  | Subida rapida y descenso | Degradacion temporal controlada.              |
| Resistencia   | 10 VU        | 30 minutos               | Estabilidad y ausencia de errores sostenidos. |
| Supabase read | Configurable | VUS/DURATION por entorno | Lectura no destructiva sin 5xx.               |

## 10. Gestion de Defectos

### 10.1. Proceso de reporte de errores

1. Registrar defecto con ID, modulo, ambiente, pasos, datos usados, resultado actual y resultado esperado.
2. Adjuntar evidencia: captura, log, reporte Vitest, Playwright, k6 o SonarQube.
3. Clasificar severidad y prioridad.
4. Asignar responsable y fecha maxima de correccion.
5. Reprobar con caso de regresion antes de cerrar.

### 10.2. Herramienta utilizada

Se recomienda GitHub Issues o tablero equivalente integrado al repositorio. Los defectos de SonarQube se gestionan desde el dashboard de Sonar; los de k6 se registran con reporte de escenario, fecha, URL, version del commit y umbrales incumplidos.

### 10.3. Priorizacion y seguimiento

| Severidad | Descripcion                                                              | Accion                         |
|-----------|--------------------------------------------------------------------------|--------------------------------|
| Critica   | Impide login, calculo economico, aprobacion, exportacion o expone datos. | Bloquea release.               |
| Alta      | Afecta flujo P0, seguridad o rendimiento esperado.                       | Corregir antes de UAT/release. |
| Media     | Afecta funcionalidad secundaria o mensajes.                              | Planificar en sprint.          |
| Baja      | Mejora visual o ajuste menor.                                            | Backlog.                       |

## 11. Riesgos y Mitigacion

### 11.1. Identificacion de riesgos

| Riesgo                                  | Impacto  | Mitigacion                                                                |
|-----------------------------------------|----------|---------------------------------------------------------------------------|
| Errores en calculos economicos          | Muy alto | Golden data, pruebas unitarias y revision del Supervisor.                 |
| Fuga de datos por IA                    | Alto     | Payload minimizado, backend proxy, prompts versionados y revision humana. |
| Pruebas de carga en ambiente incorrecto | Alto     | Ejecutar carga solo en QA; produccion solo smoke controlado.              |
| Quality Gate fallido                    | Alto     | Bloquear merge/release hasta corregir issues criticos.                    |
| Dependencias desincronizadas            | Medio    | Mantener package-lock actualizado y usar npm ci en CI.                    |
| Secretos en repositorio                 | Alto     | Gitleaks, variables de entorno y no subir .env reales.                    |
| Rutas SPA no servidas en preview        | Medio    | Configurar fallback y validar con k6 smoke.                               |

### 11.2. Impacto y probabilidad

| Riesgo                                 | Probabilidad | Impacto    | Nivel   |
|----------------------------------------|--------------|------------|---------|
| Calculo incorrecto                     | Media        | Muy alto   | Critico |
| Exposicion de evidencias               | Media        | Alto       | Alto    |
| p95 mayor a 3s bajo carga esperada     | Media        | Medio/Alto | Alto    |
| Sonar detecta vulnerabilidad critica   | Media        | Alto       | Alto    |
| Fallo de CI por credenciales faltantes | Media        | Medio      | Medio   |



### 11.3. Planes de contingencia

- Si k6 supera umbrales, aislar escenario, revisar logs, optimizar queries, cache o renderizado y repetir carga.
- Si Sonar falla, clasificar issues, corregir vulnerabilidades y duplicacion critica antes de release.
- Si la IA produce texto no sustentado, bloquear exportacion y registrar defecto IA.
- Si falla el pipeline, ejecutar localmente preflight, cobertura y build para reproducir.

## 12. Metricas y Reportes

### 12.1. Indicadores KPI de calidad

| KPI                             | Umbral                                                                | Fuente                      |
|---------------------------------|-----------------------------------------------------------------------|-----------------------------|
| Casos P0 aprobados              | 100%                                                                  | Matriz de pruebas.          |
| Defectos criticos abiertos      | 0                                                                     | GitHub Issues / tablero QA. |
| Cobertura LCOV                  | Medida y enviada a SonarQube; meta minima para logica critica >= 80%. | Vitest/V8.                  |
| Quality Gate                    | Aprobado                                                              | SonarQube.                  |
| Duplicacion en codigo nuevo     | <= 3%                                                                 | SonarQube.                  |
| Vulnerabilidades criticas/altas | 0                                                                     | SonarQube, CodeQL, ZAP.     |
| k6 http_req_failed              | < 1% en carga esperada                                                | k6.                         |
| k6 p95                          | <= 3000 ms                                                            | k6.                         |
| k6 p99                          | <= 5000 ms                                                            | k6.                         |
| Checks k6                       | > 99%                                                                 | k6.                         |

### 12.2. Informes de avance

El informe de avance debe indicar ejecucion por tipo de prueba, porcentaje de avance, defectos abiertos/cerrados, cumplimiento de umbrales k6, estado del Quality Gate, cobertura, riesgos y decisiones pendientes.

### 12.3. Reporte final de pruebas

| Seccion del reporte final | Contenido minimo                                                          |
|---------------------------|---------------------------------------------------------------------------|
| Resumen ejecutivo         | Estado general, modulos aprobados y decision go/no-go.                    |
| Ejecucion funcional       | Casos aprobados, fallidos y bloqueados.                                   |
| Rendimiento k6            | Escenario, URL, fecha, VUs, p95, p99, errores y checks.                   |
| SonarQube                 | Quality Gate, cobertura, bugs, vulnerabilites, code smells y duplicacion. |
| Seguridad                 | Hallazgos CodeQL, Gitleaks, ZAP y pruebas RBAC.                           |
| IA responsable            | Casos de prompt, revision humana y no alucinacion.                        |
| Defectos                  | Lista priorizada y estado.                                                |
| Anexos                    | Capturas, logs, reportes HTML/JSON y acta UAT.                            |

## 13. Aprobaciones

### 13.1. Responsables de validacion

| Responsable          | Rol de aprobacion                                          |
|----------------------|------------------------------------------------------------|
| Lider tecnico        | Aprueba calidad tecnica, SonarQube y cobertura.            |
| QA                   | Aprueba ejecucion de casos, defectos y evidencias.         |
| DevSecOps            | Aprueba CI/CD, secretos, k6 y seguridad.                   |
| Residente de Obra    | Aprueba utilidad del registro de metrados y expediente.    |
| Supervisor/Inspector | Aprueba fiscalizacion, observaciones y cierre del periodo. |
| Entidad/Monitor      | Aprueba consulta de solo lectura y descarga de expediente. |

### 13.2. Firmas / aceptacion del plan

| Nombre | Rol                  | Firma | Fecha |
|--------|----------------------|-------|-------|
|        | Lider tecnico        |       |       |
|        | QA                   |       |       |
|        | DevSecOps            |       |       |
|        | Residente de Obra    |       |       |
|        | Supervisor/Inspector |       |       |
|        | Representante Legal  |       |       |

## Anexo A. Comandos implementados

| Comando                   | Uso                                              |
|---------------------------|--------------------------------------------------|
| npm run quality:preflight | Valida que la configuracion k6/SonarQube exista. |
| npm run test:coverage     | Ejecuta Vitest con cobertura LCOV.               |
| npm run sonar:local       | Ejecuta SonarScanner local o Docker.             |
| npm run k6:smoke          | Prueba rapida de disponibilidad.                 |
| npm run k6:load           | Carga esperada hasta 25 VU.                      |
| npm run k6:stress         | Estres hasta 75 VU.                              |
| npm run k6:spike          | Pico hasta 60 VU.                                |
| npm run k6:soak           | Resistencia 30 minutos.                          |
| npm run k6:supabase       | Probe Supabase read-only.                        |

## Anexo B. Estado de verificacion de la implementacion

| Verificacion              | Estado                    | Observacion                                            |
|---------------------------|---------------------------|--------------------------------------------------------|
| Archivos k6 creados       | Completado                | Seis escenarios y helpers compartidos.                 |
| SonarQube configurado     | Completado                | sonar-project.properties y workflow CI.                |
| Scripts npm agregados     | Completado                | Cobertura, k6, Sonar y quality preflight.              |
| package-lock sincronizado | Completado                | Actualizado mediante npm install --package-lock-only.  |
| Preflight local           | Aprobado                  | npm run quality:preflight finalizo correctamente.      |
| Ejecucion k6 real         | Pendiente de ambiente     | Requiere k6 local, Docker o GitHub Actions con URL QA. |
| Analisis Sonar real       | Pendiente de credenciales | Requiere SONAR_TOKEN y SONAR_HOST_URL en CI/local.     |